BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

LESSON 247 · Year 13 autumn mock: Paper 2

Name: _____ Class: _____ Date: _____

Work through the pages in order. Answer in BLACK PEN first. Check in GREEN PEN only after the Exit Ticket.

Do Now · Recall previous learning

Q1. What makes a program's output testable?

Q2. What is a precondition?

Q3. Which part of a partition is guaranteed to be in its final position?

Q4. Push 4,9,2 onto a stack; pop once; enqueue the popped value to queue [7,8], then dequeue. Give remaining stack and queue.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

I Do · Read new knowledge

Key vocabulary

base case — Condition ending recursion.

trace table — Record intermediate algorithm state.

Big O — Asymptotic algorithm growth.

auxiliary space — Extra storage used by an algorithm.

New learning

Answer original practice, not OCR past paper — This diagnostic samples programming and algorithms, including learning from earlier Year 12 lessons.

Time management — Allow roughly 15 minutes per 15 marks.

Method precision — Binary search must shrink bounds, sorting traces must show complete passes, recursion must include a terminating condition.

Worked example

Unmarked warm-up: trace binary search midpoint for 7 in [1,3,5,7,9].

1. Predict the method — Check the middle 5 and discard indexes 0–2 because target 7 is larger.

2. Check and explain — Next middle is index 3 containing 7, so return 3.

We Do · Practise with guidance

Unmarked warm-up: trace first bubble pass on [5,2,4].

```
5↔2 → [2,5,4]
5↔4 → [2,4,5]
```

Predict the method

Swap 5 and 2 to get [2,5,4].

Check and explain

Swap 5 and 4. The first full pass ends at [2,4,5].

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

You Do · Work independently

BLACK PEN: Write your own answers. Do not look at the Review & Improve pages yet.

Question 1

(a) State the key precondition for binary search and explain why it is necessary. [2] (b) Trace binary search for 11 in [1,3,5,7,9,11,13], giving low/high/mid at each step and resulting index. [7] (c) Compare worst-case complexities of linear and binary searches. [6]

Question 2

(a) Starting with [5,1,4,2], show the array after ascending bubble-sort Pass 1, Pass 2 and a final no-swap checking Pass 3. [9] (b) Compare bubble sort with merge sort in worst-case time complexity and auxiliary space. [6]

Question 3

(a) Write recursive C# function SumTo(n) returning 1+…+n for n≥0, with base case. [8] (b) Trace SumTo(3) showing the recursive calls and returned value. [5] (c) State one drawback of recursive solutions. [2]

Exit Ticket · Answer before checking

A correctly written recursive function without a base case is guaranteed to terminate.

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

Review & Improve · GREEN PEN

Now change to GREEN PEN. Find each original question. Tick correct answers; read the model and rewrite mistakes.

Page 1 onwards · Do Now

Q1. Defined success criteria and unambiguous rules.

Q2. A requirement that must hold before an algorithm runs.

Q3. The pivot (under the specified partition method).

Q4. Stack bottom-to-top [4,9]; pop gives 2; queue [7,8,2] then dequeue 7 leaves [8,2].

You Do pages · Independent questions

Q1. MARK SCHEME /15. (a) Input is sorted into the ordering the algorithm expects [1]; comparisons can then discard the half that cannot contain the target [1]. (b) Start low0 high6 mid3→7 [2]; set low4 [1]; low4 high6 mid5→11 [2]; return index5 [2]. (c) Linear O(n) [2], may check all n [1]; binary O(log n) [2], halves the sorted candidate interval [1].

Q2. MARK SCHEME /15. (a) Pass 1: [1,4,2,5] [3]; pass 2: [1,2,4,5] [3]; Pass 3 compares remaining adjacent values, makes no swaps, and stays [1,2,4,5] [3]. The no-swap pass confirms completion. (b) Bubble worst O(n^2) [2]; merge sort worst O(n log n) [2]; merge generally uses O(n) auxiliary storage for merging while in-place bubble uses O(1) auxiliary [2].

Q3. MARK SCHEME /15. (a) static int SumTo(int n) [2]; if(n==0)return 0; [2]; return n+SumTo(n-1); [4] with valid braces/structure. (b) SumTo(3)=3+SumTo(2) [1], =3+2+SumTo(1) [1], =3+2+1+SumTo(0) [1], SumTo(0)=0 [1], result 6 [1]. (c) Further call-stack memory [1] and possible stack overflow on deep recursion [1].

Exit Ticket · Model answer

False. It may recurse until stack exhaustion.

Try the question again

Choose one question you got wrong. All correct? Choose the hardest.

1. Copy the whole original question:

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick and rewrite mistakes.

2. Write a complete new answer in GREEN PEN. Show working if needed: